

DSPy Learnings Report

DSPy Learnings Report

Author: Kaushal Jayapragash

Date: September 2026

Stack: Python 3.12, DSPy 3.3.0, Ollama (llama3.1:8b), OpenAI gpt-4o-mini, ChromaDB 1.5.9, SentenceTransformers 2.7.0

Overview

This report summarises a six-notebook self-directed study of [DSPy](#), Stanford NLP's framework for building and optimising LLM pipelines programmatically. The key insight DSPy offers is that LLM calls should be declared as typed programs — not hand-crafted strings — so they can be evaluated and optimised automatically.

The notebooks follow a deliberate progression:

Primitives (00) → Reasoning (01) → Composition (02) →
Evaluation (03) → Light Optimisation (04) → Full Optimisation (05) → RAG Pipeline (06)

Every notebook reuses the same `metric` → `evaluate` → `compile` loop, making the framework feel consistent regardless of pipeline complexity.

Notebook 00: Environment Setup & Core Primitives

Notebook: `00_setup_check.ipynb`

What was covered

This notebook verified the environment and introduced the two foundational abstractions in DSPy.

`dspy.LM` wraps any LLM provider. A single `dspy.configure(lm=lm)` call sets the global LM that all modules inherit.

`dspy.Signature` declares the schema of an LLM call — what goes in and what comes out — expressed as a typed Python class. It is a *contract*, not a prompt string.

dspy.Predict is the simplest module: it fulfils a signature with a single LLM call and no special reasoning strategy.

```
import dspy

lm = dspy.LM("ollama_chat/llama3.1:8b", api_base="http://localhost:11434")
dspy.configure(lm=lm)

class BasicQA(dspy.Signature):
    """Answer the question in a single, concise sentence."""
    question: str = dspy.InputField()
    answer: str = dspy.OutputField()

predict = dspy.Predict(BasicQA)
result = predict(question="What is the capital of France?")
print(result.answer)
```

Key takeaway

A Signature is a schema, not a prompt. DSPy translates the schema into a prompt automatically. This separation is what lets optimisers later rewrite the instruction text without breaking the program.

Notebook 01: ChainOfThought & Typed Outputs

Notebook: 01_cot_typed_outputs.ipynb

What was covered

This notebook introduced two features that are used in almost every subsequent pipeline: **reasoning steps** and **type-constrained outputs**.

dspy.ChainOfThought is a drop-in replacement for **dspy.Predict** that automatically inserts a reasoning field before the declared output field. The model must reason before it answers — zero extra prompt engineering required.

Typed outputs with Literal let you constrain the model's answer to a fixed set of values. DSPy injects a constraint note into the system message and retries if the model returns an invalid value.

```
from typing import Literal

class YesNo(dspy.Signature):
    """Answer the question. If uncertain, say Uncertain."""
    question: str = dspy.InputField(desc="A yes/no question")
    answer: Literal["True", "False", "Uncertain"] = dspy.OutputField()
```

```

cot      = dspy.ChainOfThought(YesNo)
result = cot(question="The smallest country in the second largest continent is North
                America.")

print(result.reasoning)  # the model's chain of thought
print(result.answer)    # constrained to True / False / Uncertain

```

`dspy.inspect_history(n=1)` was used throughout to see the raw system/user/assistant messages — an essential debugging tool.

Key takeaway

`ChainOfThought` extracts more reasoning from the model with no manual prompt work. Typed outputs turn probabilistic string generation into structured, validated data.

Notebook 02: Composing Multi-Step Modules

Notebook: `02_modules.ipynb`

What was covered

This notebook showed how to build multi-step pipelines by composing multiple signatures inside a single `dspy.Module`.

The pattern mirrors PyTorch: declare sub-modules in `__init__`, wire them together in `forward()`.

```

class QueryUnderstanding(dspy.Module):
    def __init__(self):
        self.classify = dspy.Predict(DomainClassifier)
        self.rewrite = dspy.ChainOfThought(QueryRewriter)

    def forward(self, question):
        classification = self.classify(question=question)
        rewrite = self.rewrite(question=question, domain=classification.domain)
        return dspy.Prediction(
            domain = classification.domain,
            search_query = rewrite.search_query,
            reasoning = rewrite.reasoning,
        )

```

The output of one module (`classification.domain`) flows as an input to the next, building a chain without any manual string formatting.

Key takeaway

Because sub-modules are declared in `__init__`, DSPy's optimisers can automatically discover and tune every prompt in the pipeline — not just the outermost one.

Notebook 03: Defining Metrics & Batch Evaluation

Notebook: 03_metrics.ipynb

What was covered

This notebook introduced the evaluation infrastructure that all subsequent optimisation work depends on.

dspy.Example is a labeled data point. `.with_inputs("field_name")` marks which fields are inputs to the module (vs. ground-truth labels that should not be passed in at inference time).

Metric functions follow a fixed signature: `(example, prediction, trace=None) -> float`. Returning a float between 0 and 1 makes the metric compatible with every DSPy optimiser.

dspy.evaluate.Evaluate runs a module over a dataset in parallel, aggregates metric scores, and prints a progress table.

```
from dspy.evaluate import Evaluate

def domain_accuracy(example, prediction, trace=None):
    return float(example.domain == prediction.domain)

evaluate = Evaluate(
    devset          = devset,
    metric          = domain_accuracy,
    num_threads     = 4,
    display_progress= True,
)

score = evaluate(classifier) # EvaluationResult with a .score attribute
```

Dataset used: Yahoo Answers Topics (filtered to science / sports / politics). 50 train / 50 dev.

Baseline result: 68.0% accuracy with zero-shot `dspy.Predict`.

Key takeaway

The metric is DSPy's definition of "better". Any optimiser you apply later will use this exact function to score candidate prompts. Getting the metric right is more important than getting the prompt right.

Notebook 04: BootstrapFewShot Optimisation

Notebook: 04_optimization.ipynb

What was covered

This notebook applied DSPy's simplest optimiser to automatically inject few-shot examples into the prompt.

BootstrapFewShot works in two steps: 1. Runs the module on the training set. 2. Finds examples where the model's output was correct (according to the metric), then stores those as demonstrations.

At inference time the prompt includes those demonstrations before the live question.

```
from dspy.teleprompt import BootstrapFewShot

optimizer = BootstrapFewShot(
    metric = domain_accuracy,
    max_bootstrapped_demos= 20,  # examples where the model succeeded
    max_labeled_demos = 10,  # ground-truth examples
)

optimized = optimizer.compile(Classifier(), trainset=trainset)
```

The optimised prompt visible via `dspy.inspect_history()` contained 20 Q&A pairs injected as chat turns before the live question.

Dataset: Yahoo Answers (6-domain: society / science / health / sports / entertainment / politics).
240 train / 120 dev.

Configuration	Accuracy
Zero-shot baseline	70.0%
BootstrapFewShot	74.2%
Delta	+4.2%

Key takeaway

BootstrapFewShot automates few-shot prompting. It finds examples the model already handles correctly and promotes them as in-context demonstrations — no human curation required.

Notebook 05: MIPROv2 — Full Prompt Optimisation

Notebook: 05_mipro.ipynb

What was covered

MIPROv2 is DSPy's most powerful optimiser. Unlike BootstrapFewShot (which only selects demonstrations), MIPROv2 also *writes and evaluates many candidate instruction strings*, then uses a Bayesian search (Optuna) to find the best combination of instruction + demonstrations.

```
from dspy.teleprompt import MIPROv2

optimizer = MIPROv2(metric=math_accuracy, auto='medium', verbose=True)
optimized = optimizer.compile(
    Solver(),
    trainset = trainset,
    requires_permission_to_run= False,
)

# Inspect the auto-generated instruction:
print(optimized.solve.predict.signature)
```

Auto-generated instruction (MIPROv2-written): > "When presented with a math word problem, analyse the scenario and follow a structured reasoning process to break down the calculations step by step. Compute the necessary values and derive the final numeric answer without providing supplementary explanations or units."

Hand-written original: > "Solve the math word problem. Return only the final numeric answer."

Dataset: GSM8K (grade-school math). 200 train / 100 dev. Model switched to gpt-4o-mini .

Configuration	Accuracy
Baseline (hand-written instruction, ChainOfThought)	92.0%
MIPROv2 (auto-written instruction + selected few-shots)	94.0%
Delta	+2.0%

The delta is modest because the baseline was already strong. The principle scales: on harder tasks with weaker starting prompts, MIPROv2's gains are larger.

Key takeaway

MIPROv2 does prompt engineering automatically. Given a metric and a dataset, it can write better instructions than a human hand-drafted, by exploring many candidates and measuring which wording actually improves scores on held-out data.

Notebook 06: Retrieval-Augmented Generation (RAG)

Notebook: 06_rag.ipynb

What was covered

This notebook built a full RAG pipeline in DSPy: a vector retriever feeding retrieved context into an LLM generator, all wrapped in a single module.

Retriever — a custom in-memory vector search over Wikipedia articles using SentenceTransformers embeddings and cosine similarity:

```
# L2-normalise once at startup, then dot-product == cosine similarity
corpus_embeddings /= np.linalg.norm(corpus_embeddings, axis=1, keepdims=True)

def retrieve(query: str, k: int = 3) -> list[str]:
    q_emb = embedder.encode([query])
    q_emb /= np.linalg.norm(q_emb)
    scores = corpus_embeddings @ q_emb.T
    top_k = np.argsort(scores.flatten())[-k:][::-1]
    return [corpus_texts[i] for i in top_k]
```

RAG module — wires retrieval into a ChainOfThought generator:

```
class MedicalRAG(dspy.Module):
    def __init__(self, k=3):
        self.k = k
        self.generate = dspy.ChainOfThought(GenerateAnswer)

    def forward(self, question):
        context = retrieve(question, k=self.k)
        return self.generate(context=context, question=question)
```

Metric — substring match to handle capitalisation and short-form answers:

```
def answer_match(example, prediction, trace=None):
    expected = str(example.answer).strip().lower()
    predicted = str(prediction.answer).strip().lower()
    return float(expected in predicted or predicted in expected)
```

Dataset: MedMCQA (Pharmacology subset). 150 train / 93 dev. Model: gpt-4o-mini .

Configuration	Accuracy
LM only (no RAG)	23.7%

Configuration	Accuracy
RAG (unoptimised)	8.6%
Delta	−15.1%

Why RAG underperformed: Only 9 of 25 Wikipedia articles were successfully fetched (the rest returned JSON parse errors). The thin corpus meant the retriever returned off-topic passages that distracted the model rather than helping it.

MIPROv2 optimisation was planned for this notebook but skipped due to an Optuna/NumPy version incompatibility in the local environment — the workflow was already exercised in notebook 05.

Key takeaway

Retrieval quality is a prerequisite for RAG to help. Bad context is worse than no context. DSPy makes the RAG pipeline itself modular — swap the retriever function without touching the generator signature — but it cannot compensate for a thin or noisy knowledge base.

Summary of DSPy Concepts

Concept	Introduced In	Purpose
<code>dspy.Signature</code>	00	Typed schema for an LLM call (input + output fields)
<code>dspy.Predict</code>	00	Single-call module; simplest execution strategy
<code>dspy.configure(lm=...)</code>	00	Global LM; inherited by all modules
<code>dspy.ChainOfThought</code>	01	Adds auto-inserted <code>reasoning</code> field before output
<code>Literal</code> typed outputs	01	Constrains model output to a fixed set of values
<code>dspy.inspect_history()</code>	01	Prints raw prompt/completion for debugging
<code>dspy.Module</code> + <code>forward()</code>	02	Composable multi-step pipelines
<code>dspy.Prediction</code>	02	Named-field return type for <code>forward()</code>

Concept	Introduced In	Purpose
<code>dspy.Example + .with_inputs()</code>	03	Labeled data points for evaluation / optimisation
<code>Metric (example, pred) -> float</code>	03	Universal "better" definition; used by all optimisers
<code>dspy.evaluate.Evaluate</code>	03	Parallel batch evaluation with progress reporting
<code>BootstrapFewShot</code>	04	Automatic few-shot demonstration selection
MIPROv2	05	Bayesian search over instruction text + demonstrations
RAG module pattern	06	Retrieval-augmented generation pipeline

Key Takeaways Across the Series

1. **Signatures separate schema from strategy.** Declaring inputs/outputs as typed fields — not prompt strings — is what makes DSPy programs optimisable.
2. **The metric is the most important thing you write.** Every optimiser uses the metric to decide what "better" means. Investing in a good metric pays dividends across every subsequent experiment.
3. **Optimisation is compilation.** `optimizer.compile(module, trainset=...)` returns a new module with better prompts baked in. The original module is unchanged; you can swap versions easily.
4. **Composition scales.** Because sub-modules are declared in `__init__`, an optimiser applied to a parent module can tune every nested sub-module's prompt automatically.
5. **RAG is not always better.** Retrieval quality gates everything. A strong parametric LM may outperform RAG over a thin or noisy knowledge base — notebook 06 demonstrated this concretely.
6. **The framework is consistent.** The same `Evaluate + metric + compile` pattern appears in every notebook from 03 onward, regardless of whether the task is classification, math, or open-domain QA.